HTTPS

Ingeniería de Sistemas

Universidad Austral — 2026

Así viaja un *request* HTTP

```
POST /login HTTP/1.1
Host: snippets.duckdns.org
Content-Type: application/json
Cookie: sesion=7f3a9c

{"user": "ana", "password": "1234"}
```

Es **texto plano**. Lo puede leer cualquiera en el camino: el wifi del bar, el *router*, tu proveedor de internet.

- La **contraseña**, tal cual
- La ***cookie* de sesión**: con eso se hacen pasar por vos

¿Qué es HTTPS?

- Versión segura de HTTP
- Comunicación **cifrada**
- Usa el protocolo **TLS**
- Permite **verificar la identidad** del proveedor

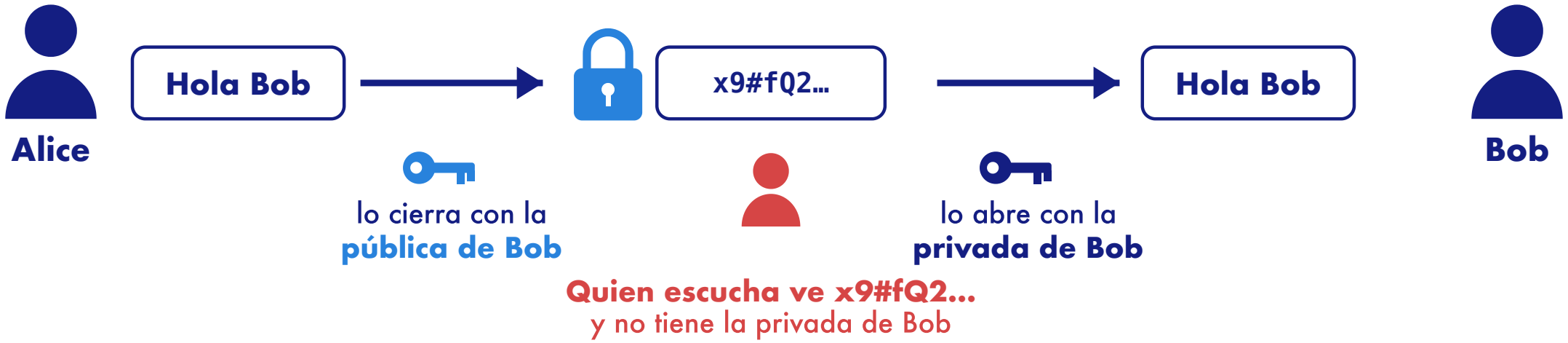


Prerrequisitos

Hay que confiar en que la criptografía de clave pública y las firmas digitales funcionan:

1. Cualquier mensaje cifrado con la **clave pública de Bob** solo se puede descifrar con la **clave privada de Bob**.
2. Cualquiera que tenga la **clave pública de Alice** puede verificar que un mensaje (firma) solo pudo haberlo creado alguien con acceso a la **clave privada de Alice**.

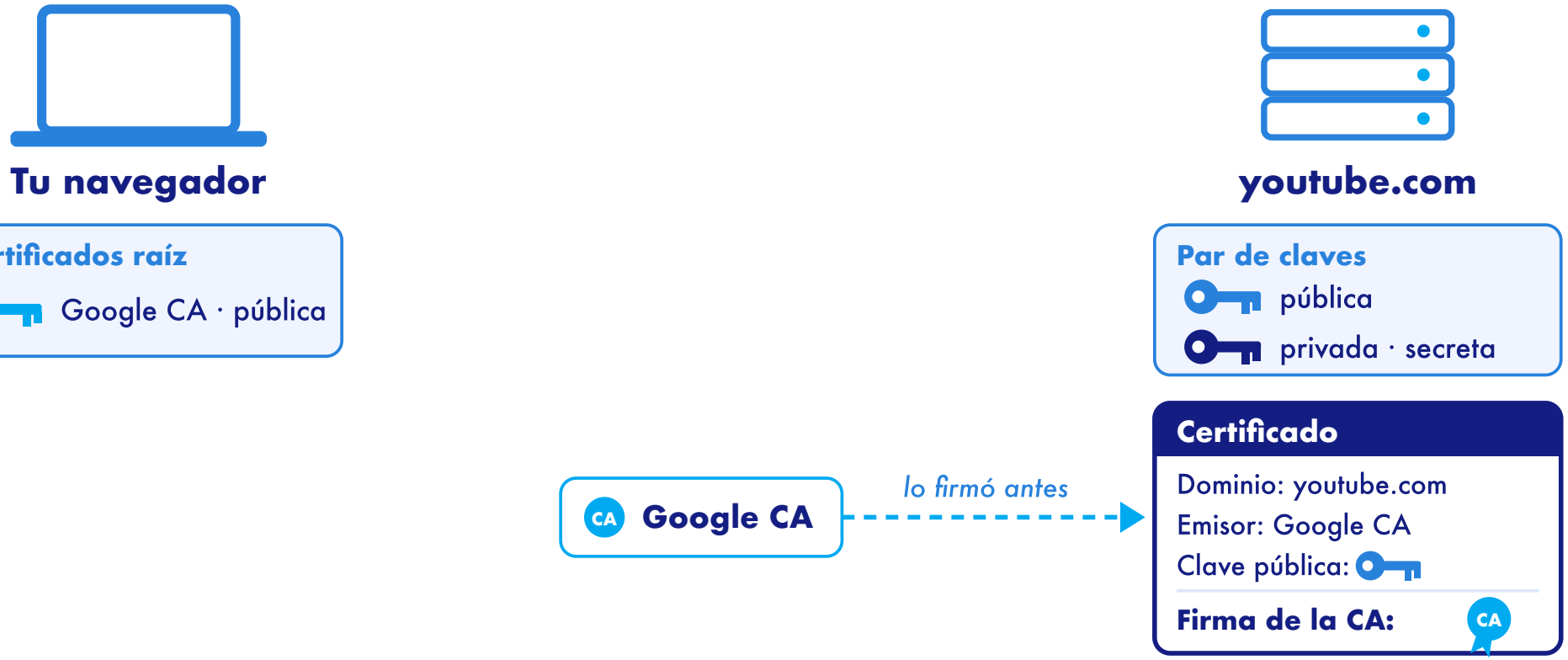
Confidencialidad: solo Bob lo puede leer



Autenticidad: solo Alice pudo firmarlo



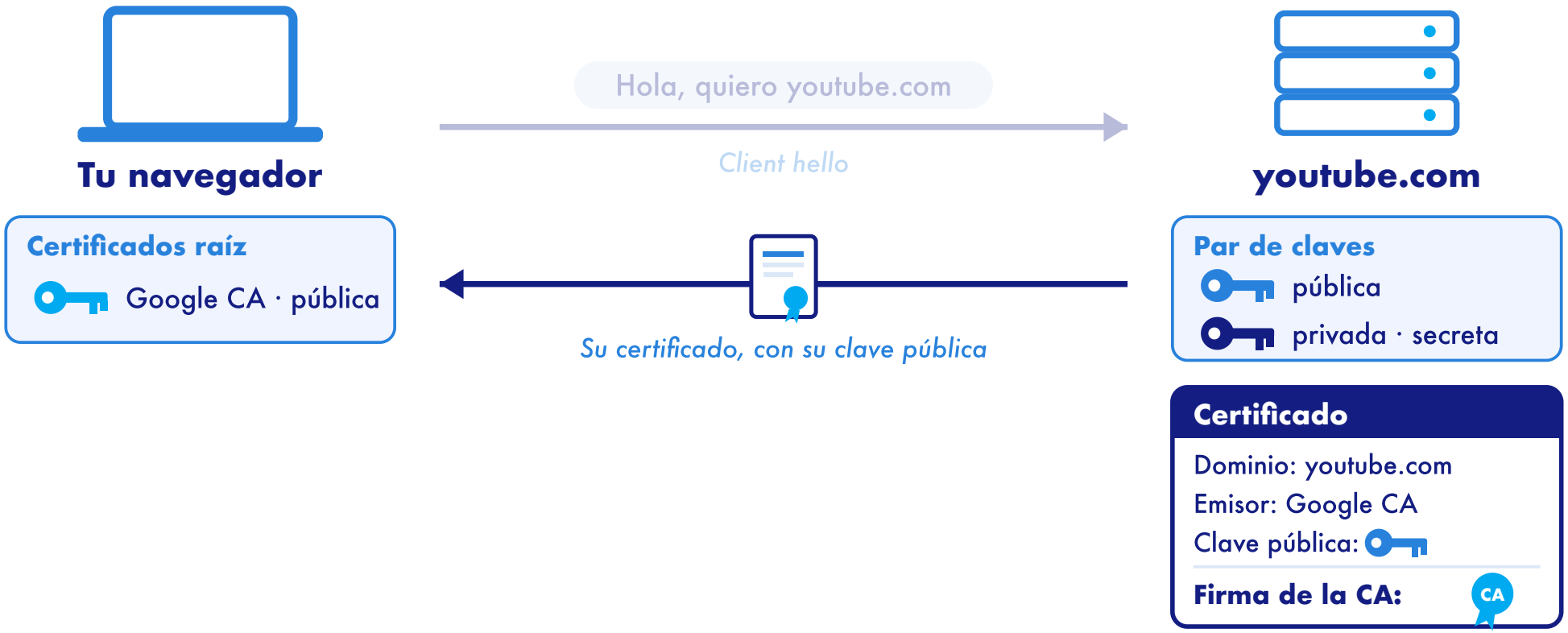
Cómo se establece la conexión



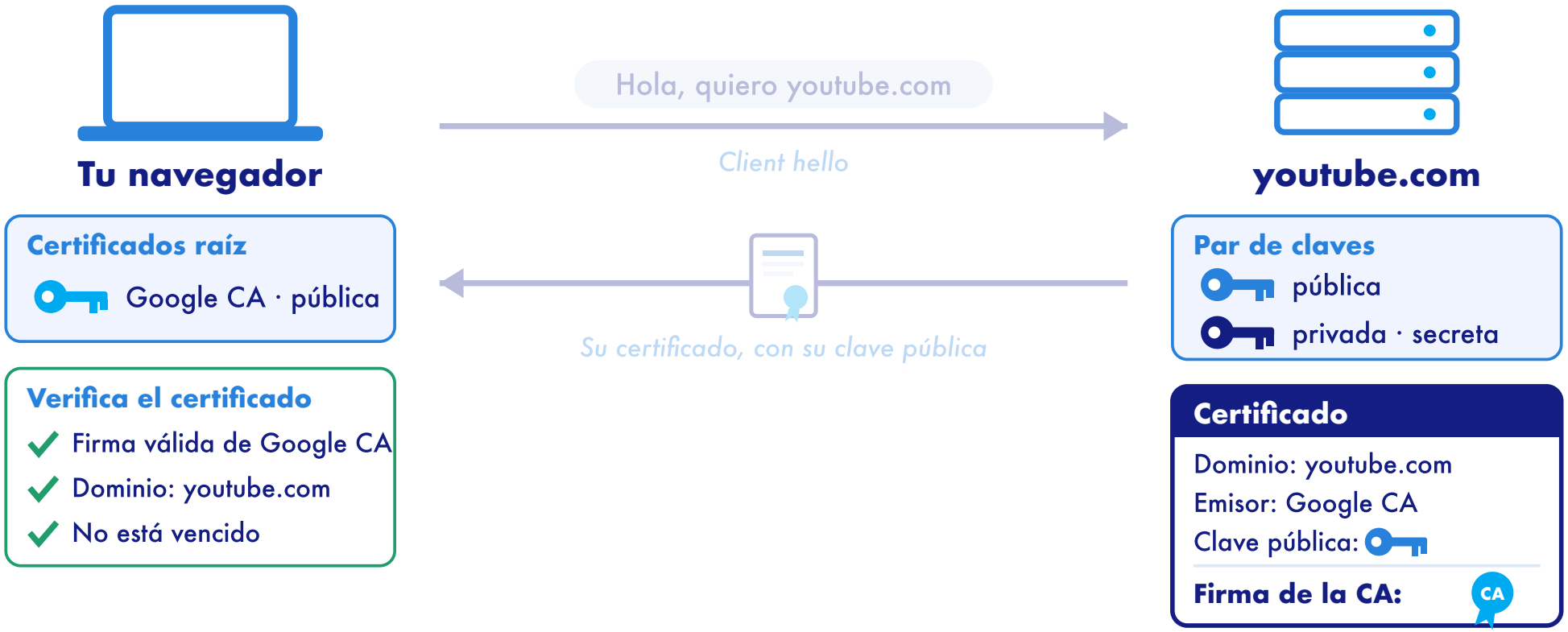
1 · El navegador saluda



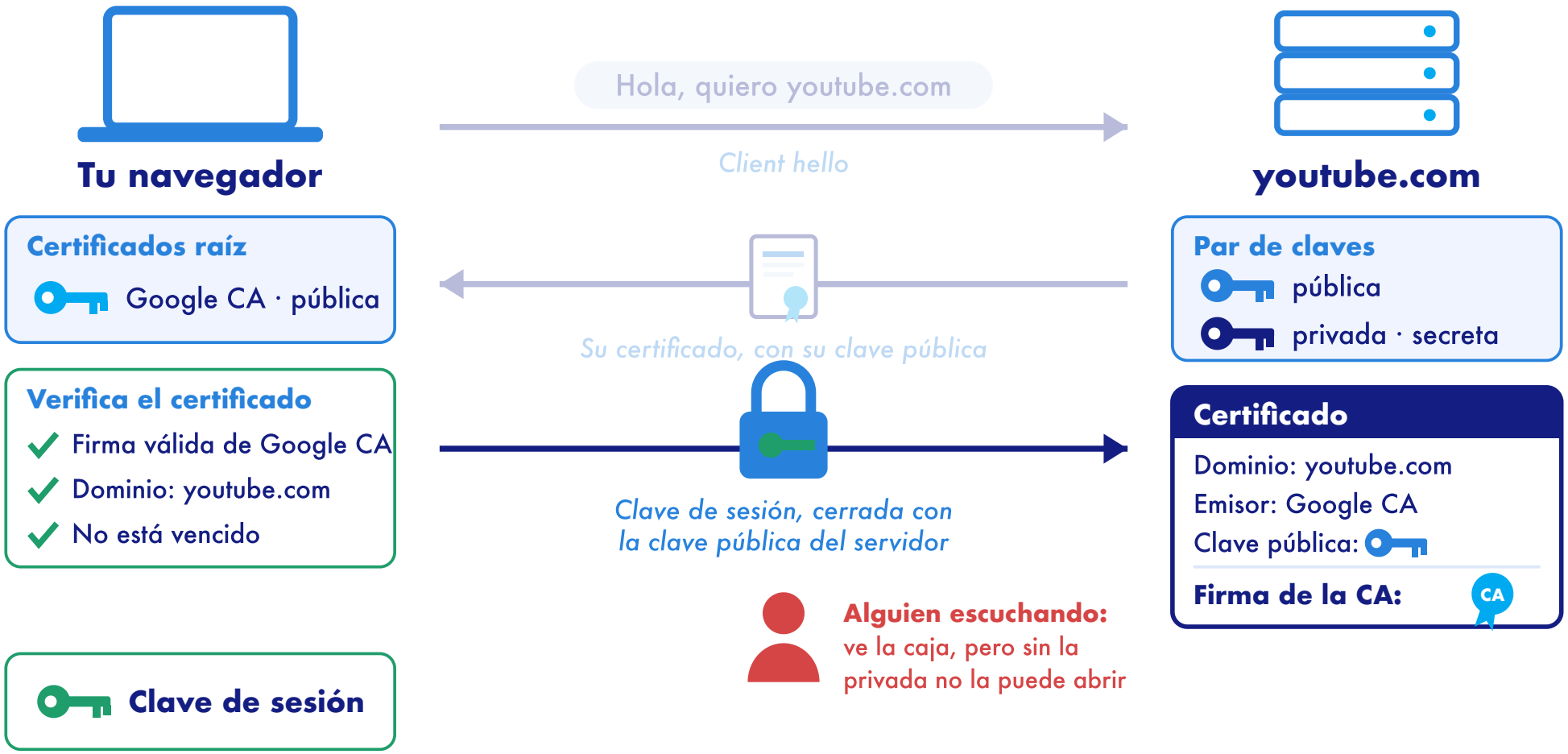
2 · El servidor presenta su certificado



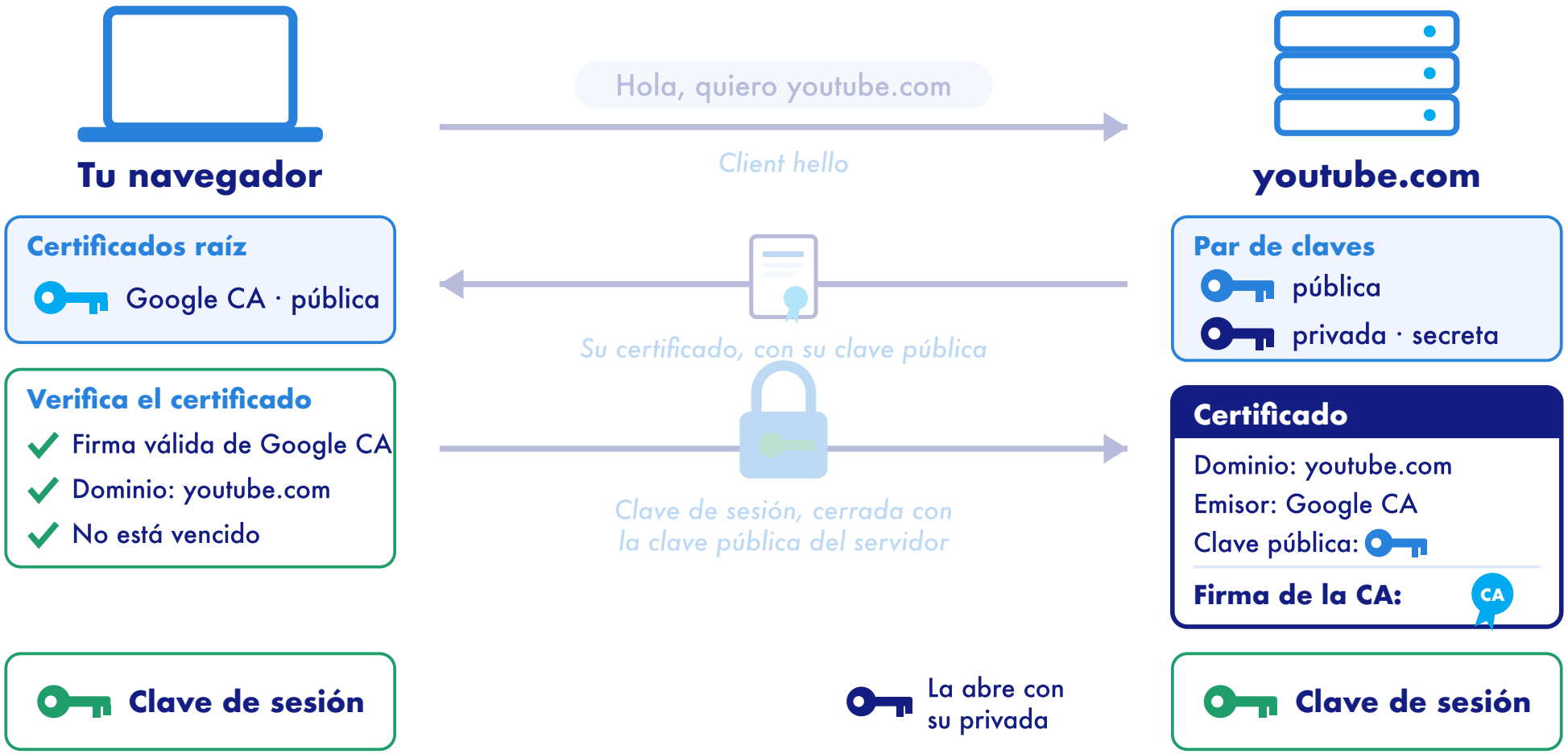
3 · El navegador verifica el certificado



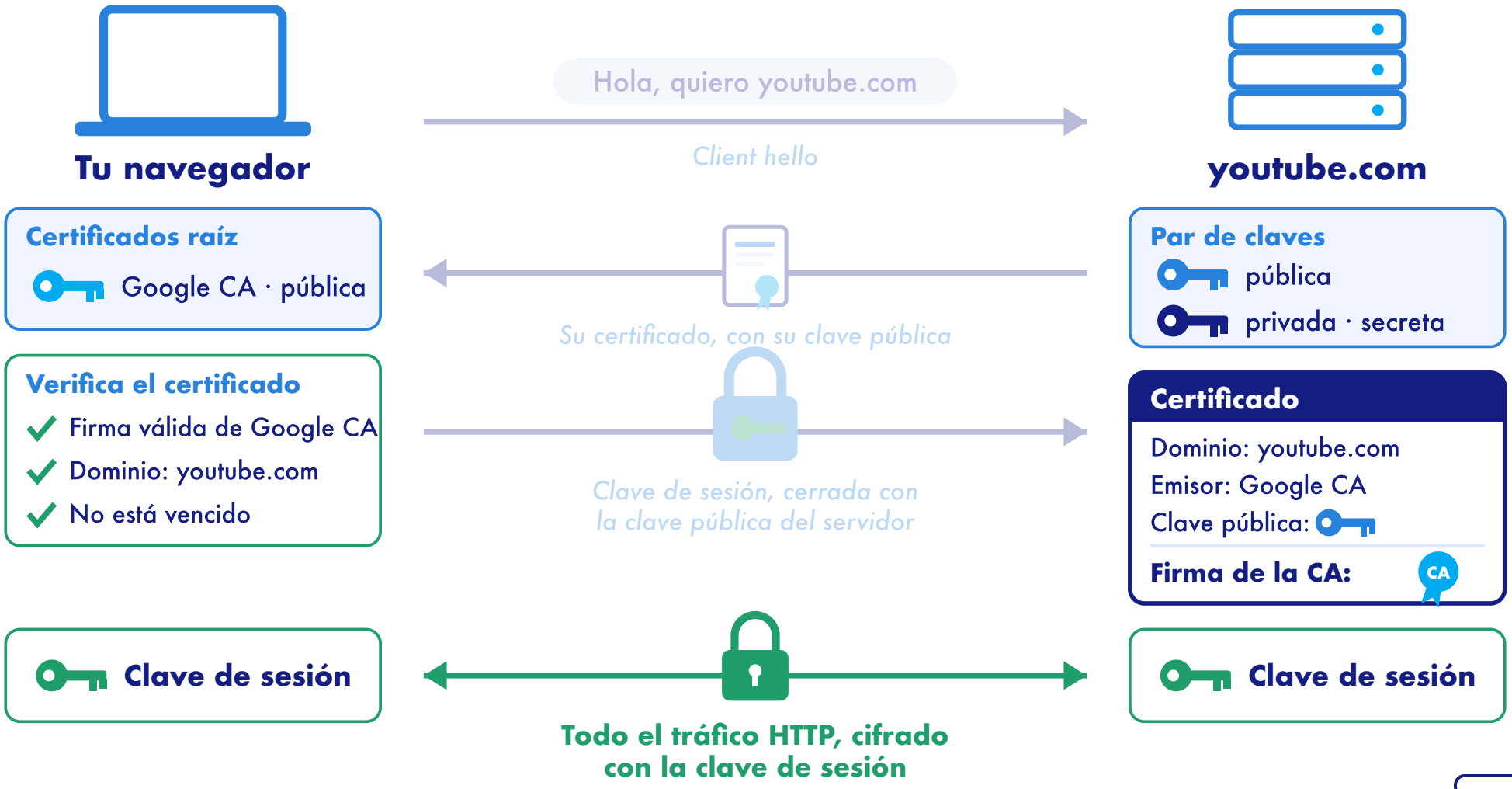
4 · El navegador manda una clave secreta



5 · El servidor la abre con su clave privada



6 · A partir de acá, todo cifrado



Transport Layer Security (TLS)

- Es la evolución de *Secure Sockets Layer* (SSL): la primera versión de TLS se basa en SSL 3.1

Aporta

- **Cifrado**
- **Autenticación**
- **Integridad**

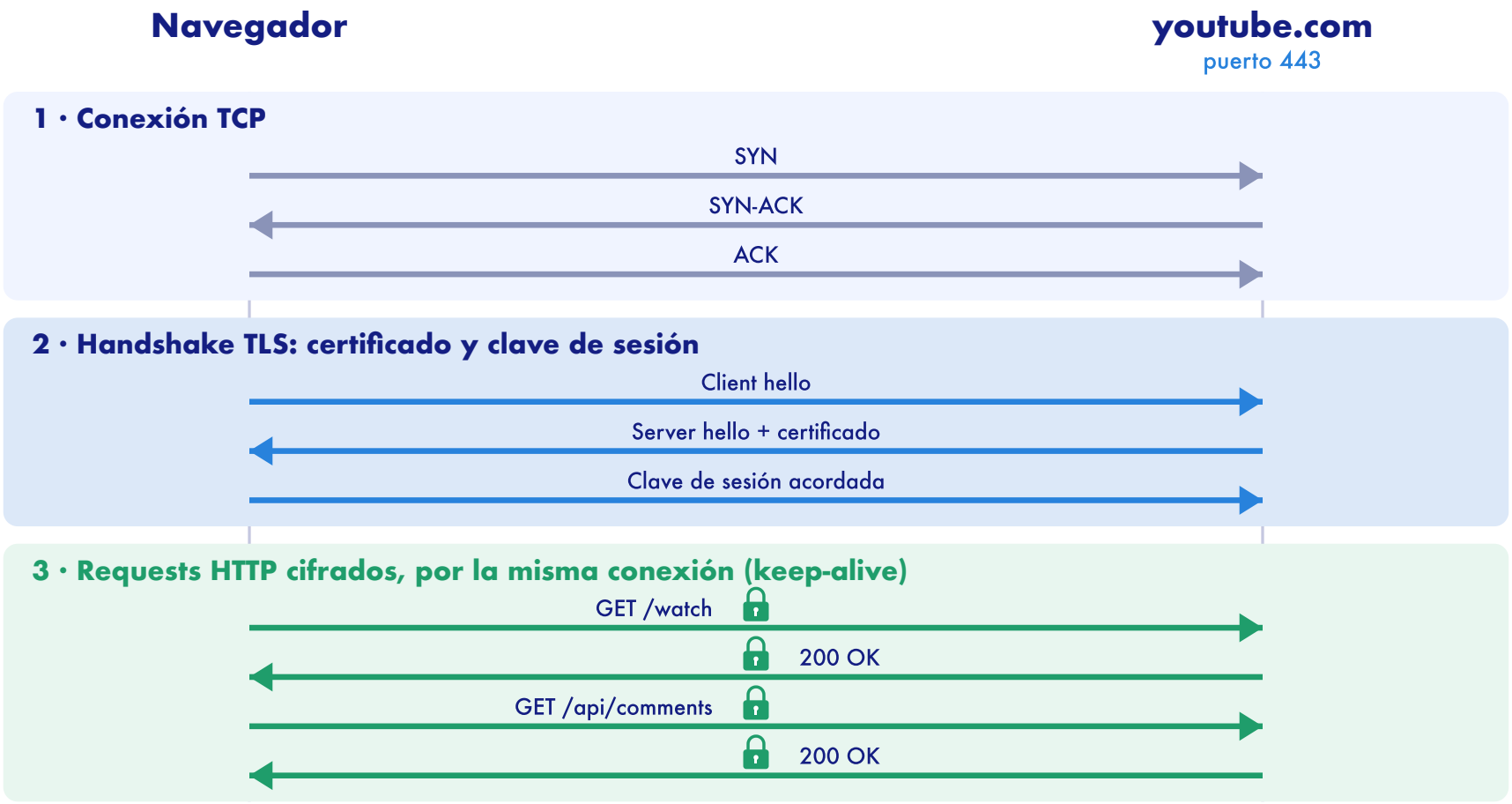
Requiere

- Certificado TLS
- Claves de cifrado asimétrico
- *Handshake*
- Generación de claves de sesión

HTTP y HTTPS en capas

Modelo OSI	HTTP	HTTPS
7 Aplicación	**HTTP**	**HTTP**
6 Presentación	**Nada** los datos viajan en claro	**TLS** cifra · autentica · verifica
5 Sesión		
4 Transporte	**TCP** puerto 80	**TCP** puerto 443
3 Red	**IP**	**IP**
2 Enlace	**Ethernet / Wi-Fi**	**Ethernet / Wi-Fi**
1 Física		
Lo que ve alguien escuchando la red:	`POST /login pass=1234`	`17 03 03 00 4a 8f e1 9c 2b …`

Qué pasa en un *request* HTTPS



Certificado TLS/SSL

- El **dominio** para el que se emitió
- Subdominios asociados
- A qué persona, organización o dispositivo se le emitió
- Qué **autoridad certificante** lo emitió
- Fecha de emisión
- Fecha de **vencimiento**
- La **clave pública**
- La **firma digital** de la autoridad certificante



Certificate Authority (CA)

Es una empresa u organización que emite certificados digitales. Esos certificados validan la identidad de las entidades y las vinculan con claves criptográficas.

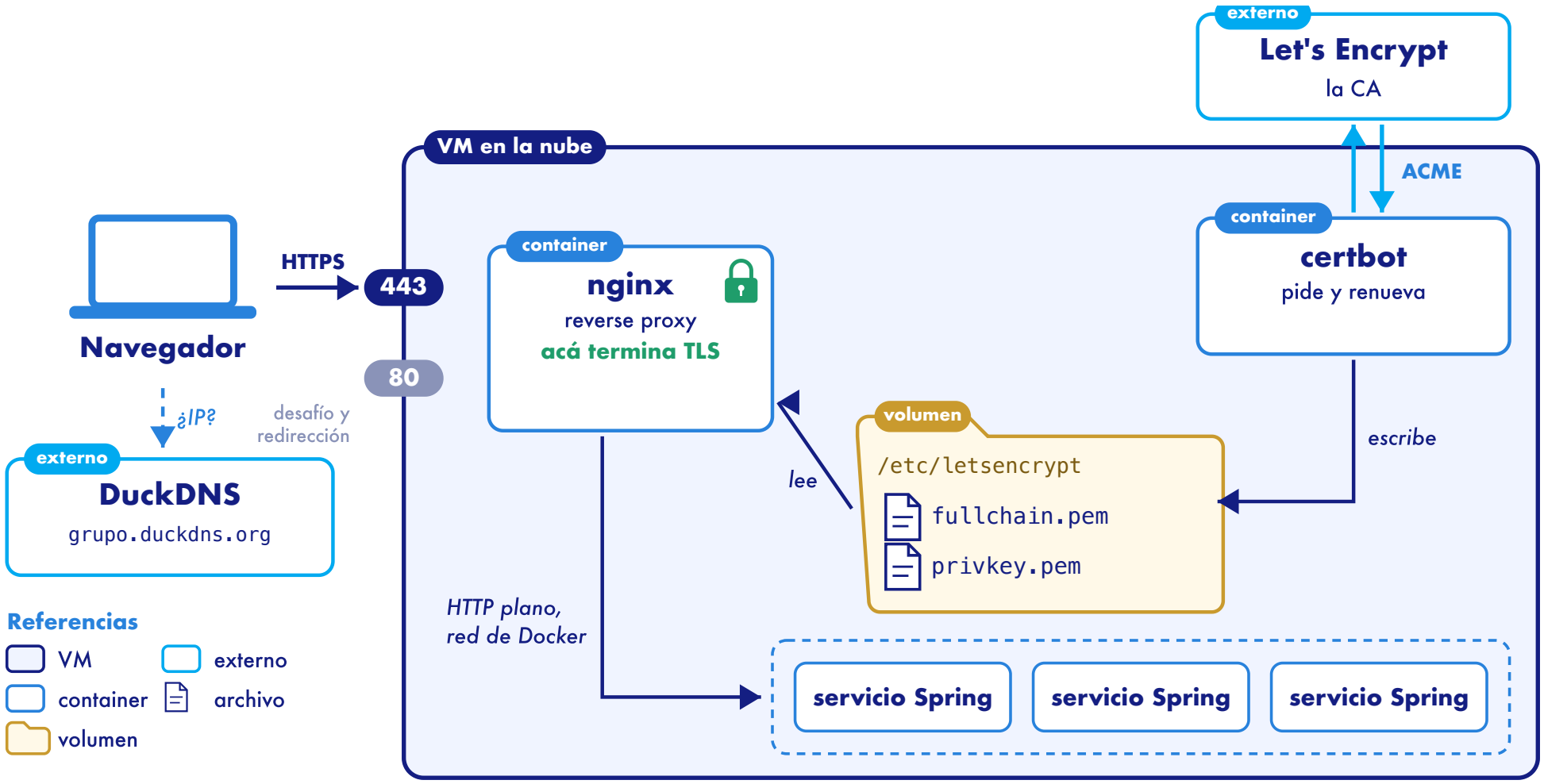
¿Cómo validan los certificados los navegadores?

- **Certificados raíz** (*root certificates*)
- Ejemplo: certificados válidos para Firefox

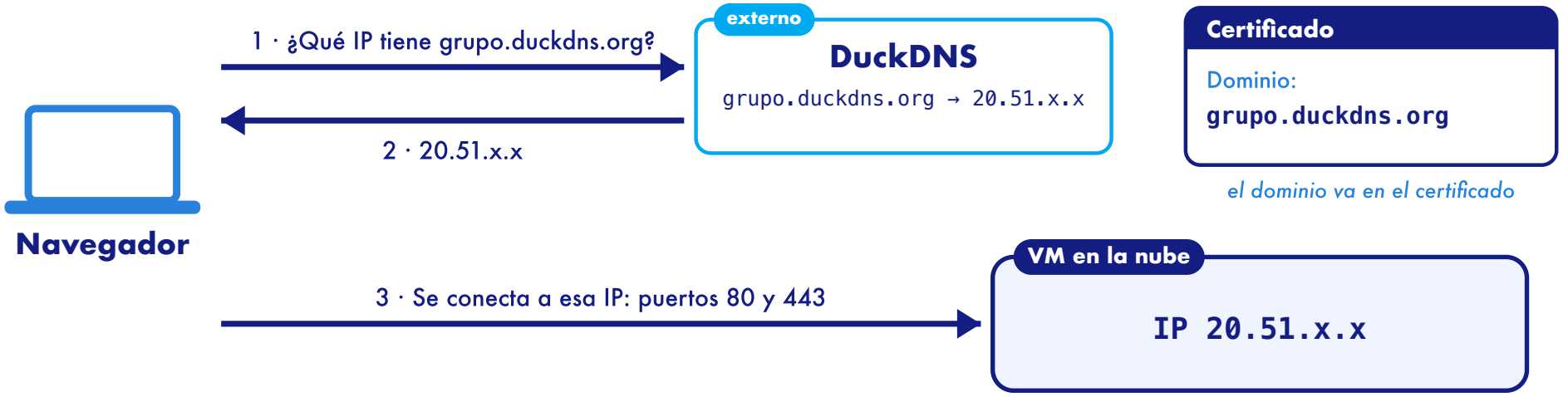
HTTPS en el TP

Let's Encrypt + DuckDNS

El panorama

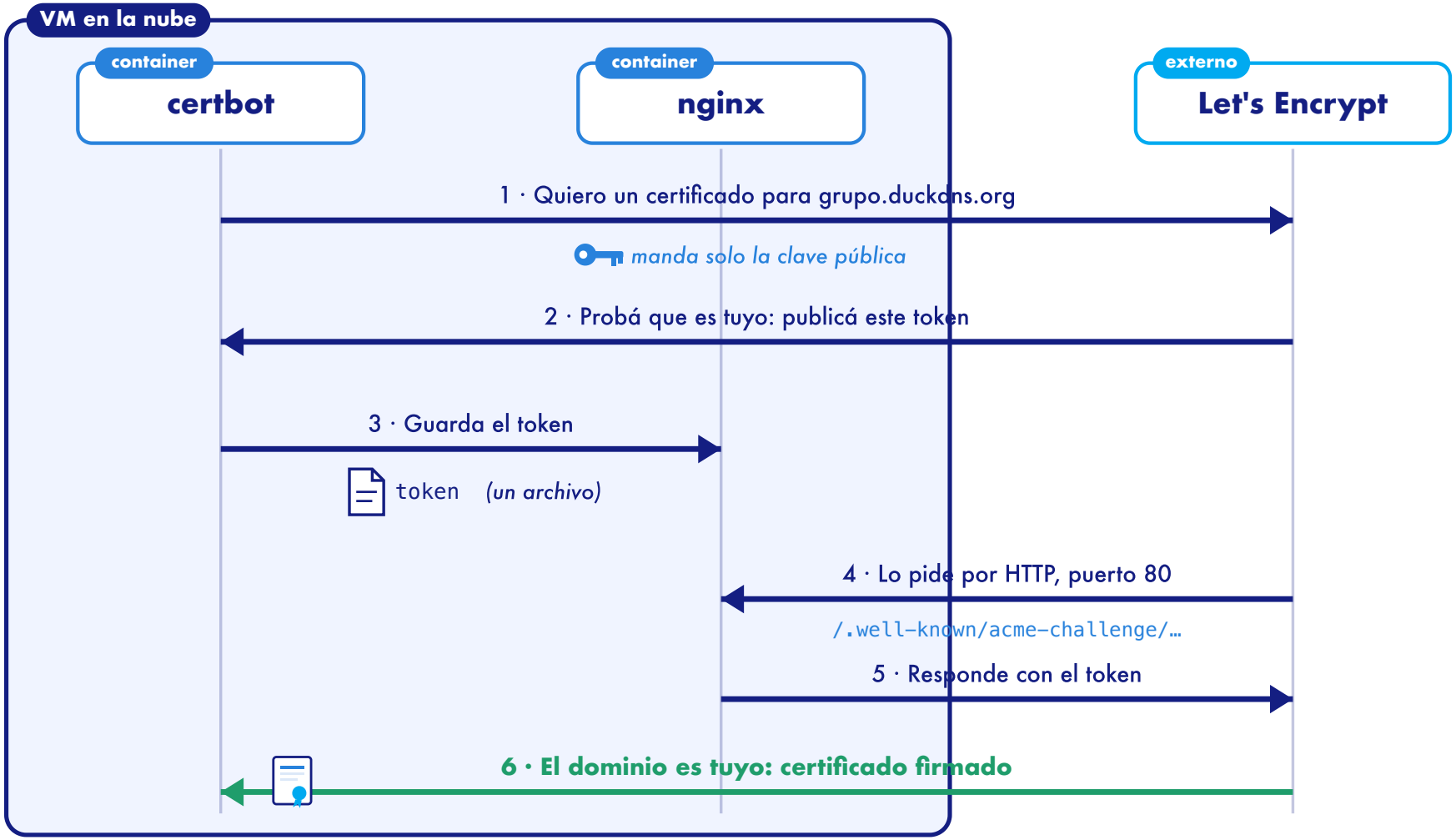

externo
Let's Encrypt
la CA
ACME
VM en la nube
container
certbot
pide y renueva
escribe
container
nginx
reverse proxy
acá termina TLS
HTTPS
443
80
Navegador
¿IP?
desafío y
redirección
externo
DuckDNS
grupo.duckdns.org
volumen
/etc/letsencrypt
fullchain.pem
privkey.pem
lee
HTTP plano,
red de Docker
servicio Spring
servicio Spring
servicio Spring
Referencias
VM
externo
container
archivo
volumen

Paso 1 · Un dominio con DuckDNS

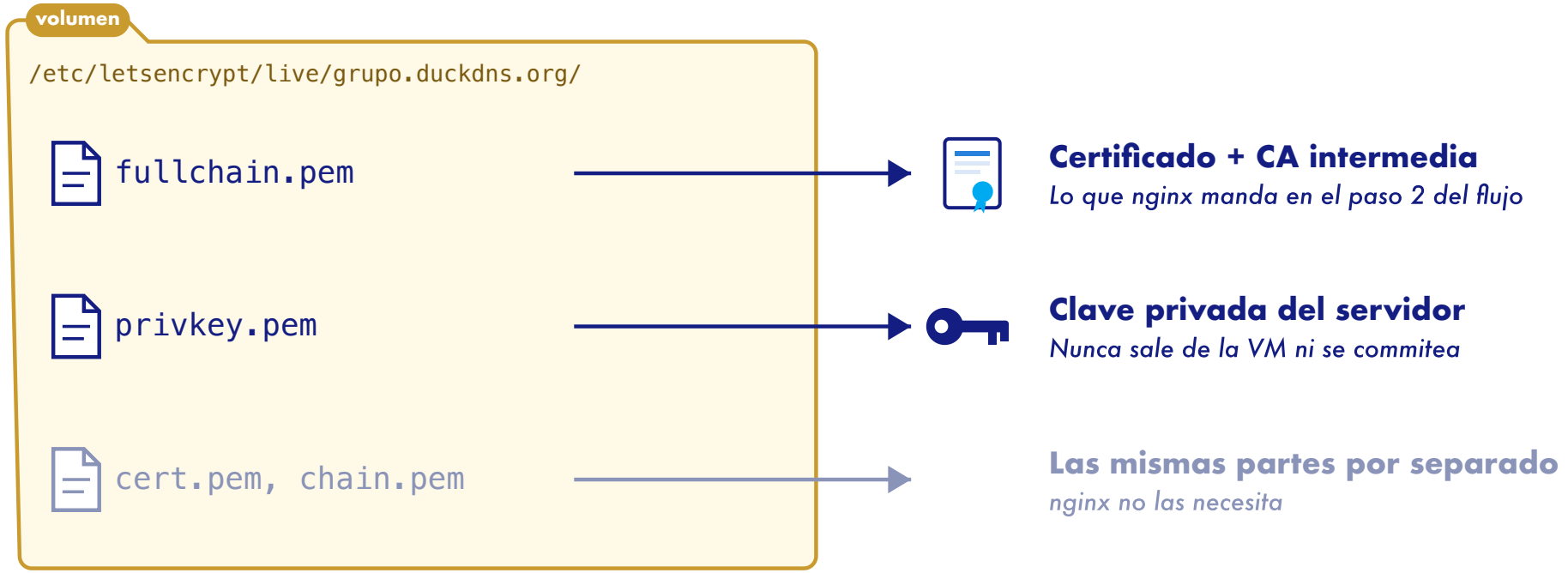


- Crear un subdominio en duckdns.org y apuntarlo a la IP pública de la VM. Uno por ambiente: `grupo-dev` y `grupo-prod`
- Let's Encrypt **no firma certificados para una IP**: el campo "Dominio" del certificado necesita un nombre

Paso 2 · Let's Encrypt verifica que el dominio es tuyo



Paso 3 · Los archivos que deja certbot



Certificado + CA intermedia
Lo que nginx manda en el paso 2 del flujo

Clave privada del servidor
Nunca sale de la VM ni se commitea

Las mismas partes por separado
nginx no las necesita

Paso 4 · Conectarlo en Docker Compose

```yaml
services:
  nginx:
    image: nginx:1.27
    ports:
      - "80:80"     # desafío de Let's Encrypt y redirección a HTTPS
      - "443:443"   # HTTPS
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf   # archivo de configuración
      - ./certbot/conf:/etc/letsencrypt                        # volumen: certificado y clave
      - ./certbot/www:/var/www/certbot                         # volumen: tokens del desafío

  certbot:
    image: certbot/certbot
    volumes:
      - ./certbot/conf:/etc/letsencrypt
      - ./certbot/www:/var/www/certbot
```

Los dos *containers* montan **las mismas carpetas**: certbot escribe, nginx lee.

Paso 4 · Conectarlo en nginx

```
server {
    listen 80;
    server_name grupo.duckdns.org;

    location /.well-known/acme-challenge/ { root /var/www/certbot; }   # paso 2: el token
    location / { return 301 https://$host$request_uri; }                # todo lo demás, a HTTPS
}

server {
    listen 443 ssl;
    server_name grupo.duckdns.org;

    ssl_certificate     /etc/letsencrypt/live/grupo.duckdns.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/grupo.duckdns.org/privkey.pem;

    # acá sigue la configuración del reverse proxy hacia los servicios
}
```

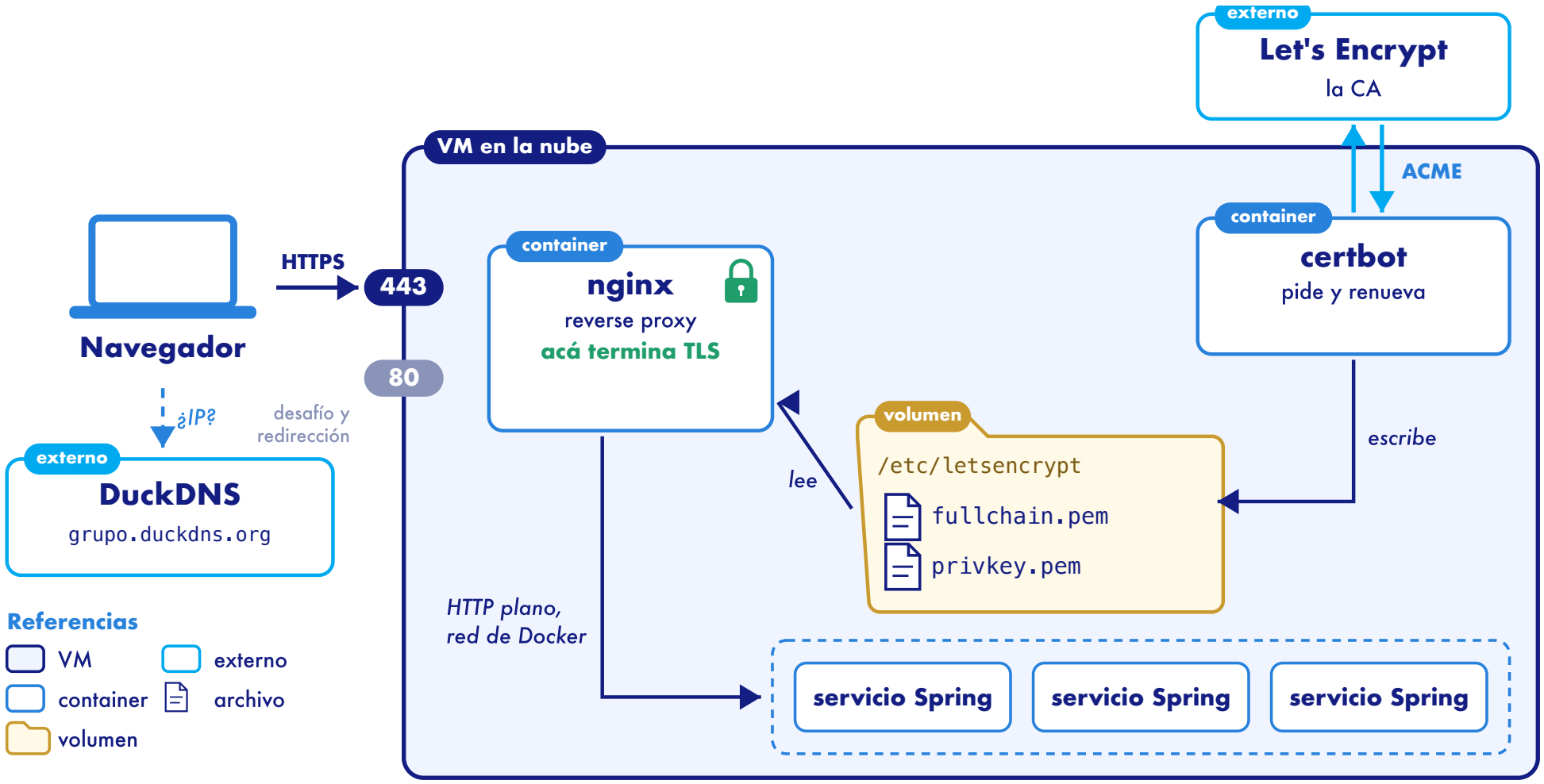
Paso 5 · Renovar



Después de renovar: recargar nginx para que lea el archivo nuevo

- Un *container* de certbot que corra `certbot renew` periódicamente
- nginx lee el certificado al arrancar: después de renovar, `nginx -s reload`

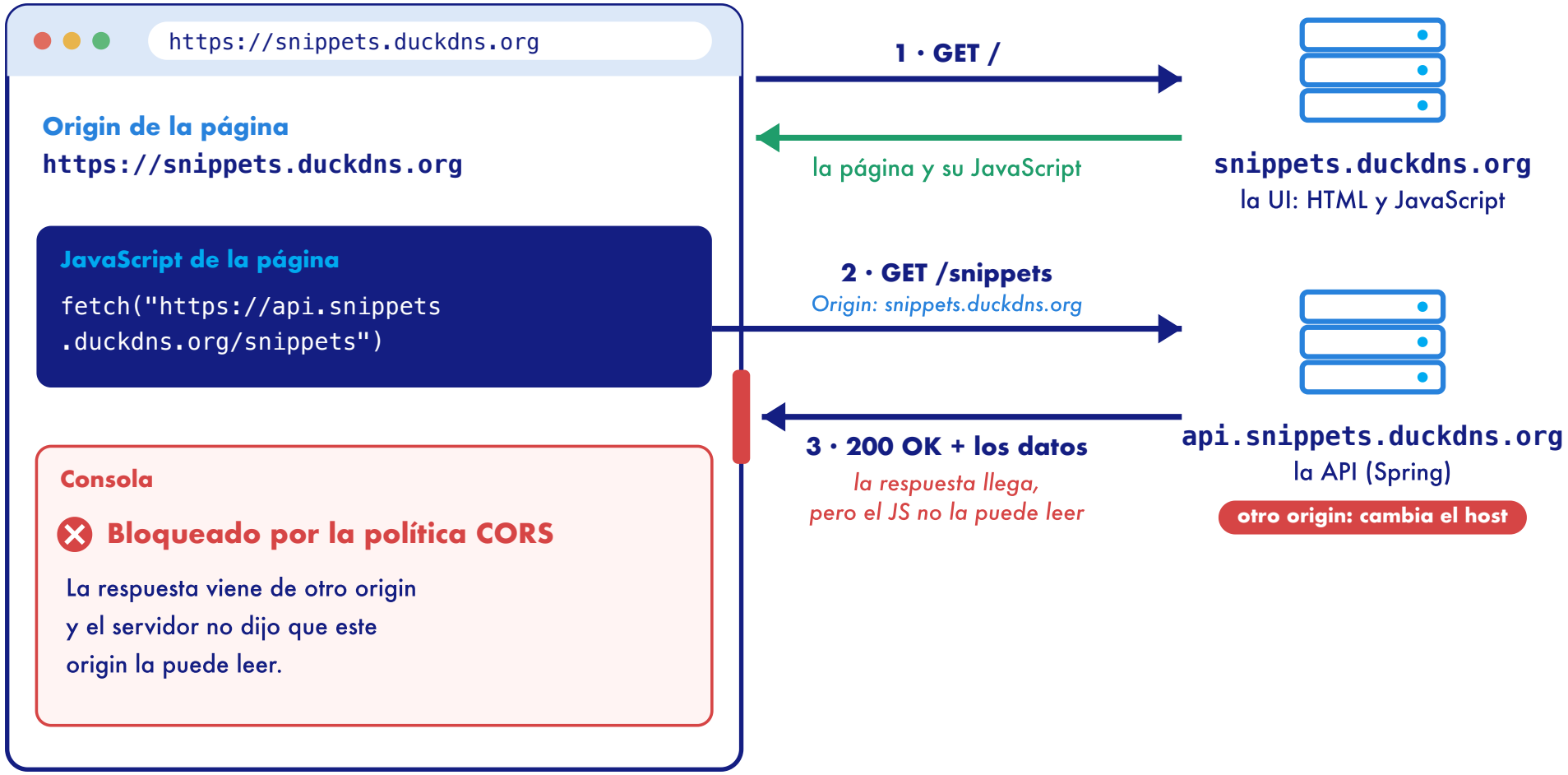
El panorama, de nuevo



CORS

Cross-Origin Resource Sharing

El problema


https://snippets.duckdns.org
Origin de la página
https://snippets.duckdns.org
JavaScript de la página
fetch("https://api.snippets
.duckdns.org/snippets")
Consola
Bloqueado por la política CORS
La respuesta viene de otro origin
y el servidor no dijo que este
origin la puede leer.
1 · GET /
la página y su JavaScript
snippets.duckdns.org
la UI: HTML y JavaScript
2 · GET /snippets
Origin: snippets.duckdns.org
3 · 200 OK + los datos
la respuesta llega,
pero el JS no la puede leer
api.snippets.duckdns.org
la API (Spring)
otro origin: cambia el host

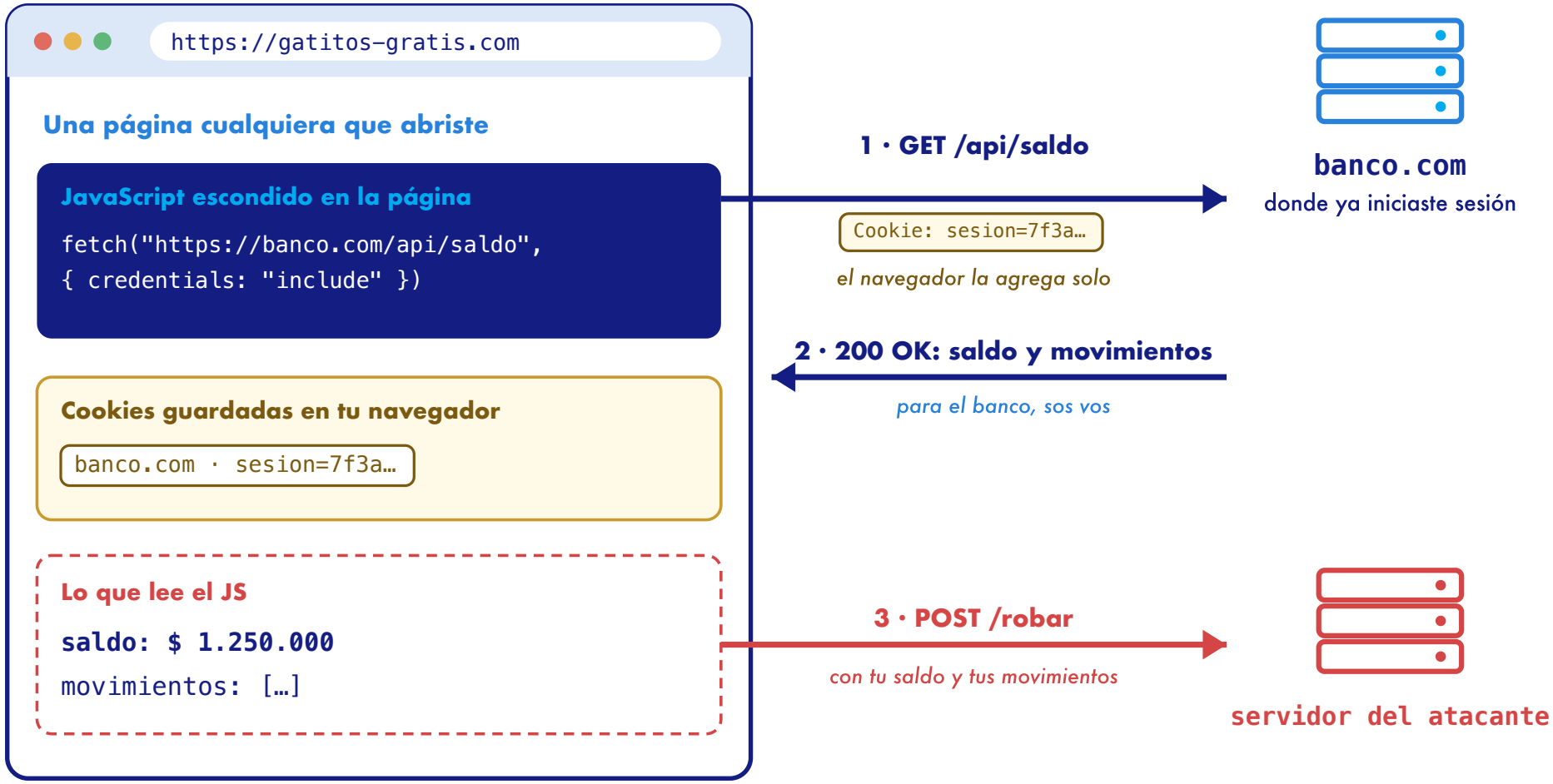
Same-origin policy

El navegador solo deja que el JavaScript de una página lea respuestas de su **mismo *origin***: **esquema + *host* + puerto**.

Si la UI vive en `https://snippets.duckdns.org`:

***Request* a**	**¿Mismo *origin*?**
`https://snippets.duckdns.org/api/snippets`	✅ Sí
`http://snippets.duckdns.org/api/snippets`	❌ Cambia el esquema
`https://api.snippets.duckdns.org/snippets`	❌ Cambia el *host*
`https://snippets.duckdns.org:8080/snippets`	❌ Cambia el puerto

Si no existiera la *same-origin policy*



Con la *same-origin policy*



¿Qué es CORS?

Un mecanismo basado en *headers* HTTP con el que **el servidor declara qué otros *origins* pueden leer sus respuestas**.

- Lo **aplica el navegador**, no el servidor: el servidor solo responde *headers*
- Por eso con `curl` o Postman "anda" y desde la UI no: ninguno de los dos es un navegador
- No es una medida de seguridad del servidor: **no reemplaza la autenticación**, protege al usuario del navegador

Preflight · el navegador pregunta antes



Navegador

`snippets.duckdns.org`

El JS quiere mandar

`POST /snippets`

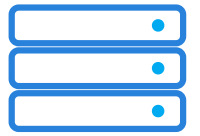
`Authorization: Bearer …`

1 · Antes, pregunta: OPTIONS /snippets

`Origin: https://snippets.duckdns.org`

`Access-Control-Request-Method: POST`

`Access-Control-Request-Headers: authorization`



`api.snippets.duckdns.org`

la API (Spring)

Preflight · el servidor responde



Navegador

`snippets.duckdns.org`

El JS quiere mandar

`POST /snippets`

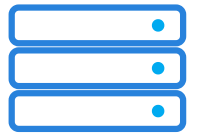
`Authorization: Bearer …`

1 · Antes, pregunta: OPTIONS /snippets

```
Origin: https://snippets.duckdns.org
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization
```

2 · 204: está permitido

```
Access-Control-Allow-Origin: https://snippets…
Access-Control-Allow-Methods: GET, POST, …
Access-Control-Allow-Headers: authorization, …
```



`api.snippets.duckdns.org`

la API (Spring)

Preflight · el navegador compara



Navegador

`snippets.duckdns.org`

El JS quiere mandar

```
POST /snippets
Authorization: Bearer …
```

Compara

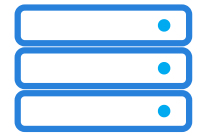
- ✓ Origin permitido
- ✓ POST permitido
- ✓ Authorization permitido

1 · Antes, pregunta: OPTIONS /snippets

```
Origin: https://snippets.duckdns.org
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization
```

2 · 204: está permitido

```
Access-Control-Allow-Origin: https://snippets…
Access-Control-Allow-Methods: GET, POST, …
Access-Control-Allow-Headers: authorization, …
```



`api.snippets.duckdns.org`

la API (Spring)

Preflight · recién ahí, el *request* real



Navegador

`snippets.duckdns.org`

El JS quiere mandar

```
POST /snippets
Authorization: Bearer …
```

Compara

- ✓ Origin permitido
- ✓ POST permitido
- ✓ Authorization permitido

✓ **El JS lee la respuesta**

1 · Antes, pregunta: OPTIONS /snippets

```
Origin: https://snippets.duckdns.org
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization
```

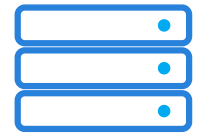
2 · 204: está permitido

```
Access-Control-Allow-Origin: https://snippets…
Access-Control-Allow-Methods: GET, POST, …
Access-Control-Allow-Headers: authorization, …
```

3 · Ahora sí: el POST real

4 · 201 Created

```
Access-Control-Allow-Origin: https://snippets…
```



`api.snippets.duckdns.org`

la API (Spring)

Preflight · si el servidor no lo permite



Navegador

`snippets.duckdns.org`

El JS quiere mandar

```
POST /snippets
Authorization: Bearer …
```

Compara

- El servidor no lo permite

1 · Antes, pregunta: OPTIONS /snippets

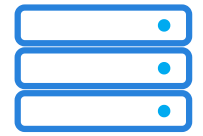
```
Origin: https://snippets.duckdns.org
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization
```

2 · 204, pero sin los headers de CORS

```
(ningún Access-Control-Allow-…)
```

3 · El POST real nunca sale

el servidor ni se entera



`api.snippets.duckdns.org`

la API (Spring)

Los *headers* que importan

Header (respuesta)	Para qué
`Access-Control-Allow-Origin`	Qué *origin* puede leer la respuesta
`Access-Control-Allow-Methods`	Qué métodos acepta (en el *preflight*)
`Access-Control-Allow-Headers`	Qué *headers* puede mandar el cliente
`Access-Control-Allow-Credentials`	Si se aceptan *cookies* o credenciales
`Access-Control-Max-Age`	Cuánto tiempo se cachea el *preflight*

El navegador manda `Origin` en cada *request cross-origin*.

Habilitarlo en Spring Boot

```kotlin
@Bean
fun corsConfigurationSource(): CorsConfigurationSource {
    val config = CorsConfiguration().apply {
        allowedOrigins = listOf("https://snippets.duckdns.org", "http://localhost:5173")
        allowedMethods = listOf("GET", "POST", "PUT", "DELETE", "OPTIONS")
        allowedHeaders = listOf("Authorization", "Content-Type")
    }
    return UrlBasedCorsConfigurationSource().apply {
        registerCorsConfiguration("/**", config)
    }
}
```

Con Spring Security hay que activarlo también en la cadena de filtros: `http.cors { }`. Si no, el *preflight* llega sin *token* y responde `401`.

Referencias

- Video de kubucation (YouTube)
- What is HTTPS? — Cloudflare
- What happens in a TLS handshake? — Cloudflare
- How It Works — Let's Encrypt
- Challenge Types — Let's Encrypt
- DuckDNS
- Cross-Origin Resource Sharing (CORS) — MDN
- CORS — Spring Security